

Basics

Topics covered: Program Structure, Output, Input (Scanner), Variables & Data Types, Type Casting, Operators (all types), Operator Precedence, Math Class Functions, Escape Sequences, Comments & Constants.

Part 1 — Notes

1. Java Program Structure

```
public class Main {  
    public static void main(String[] args) {  
        // code goes here  
    }  
}
```

main() must be exactly public static void main(String[] args) — fixed JVM entry point signature.

2. Output

System.out.println() — prints, then moves to new line.

System.out.print() — prints, no new line.

System.out.printf("%.2f", value) — formatted output, e.g. 2 decimal places.

System.out.printf("%d and %s", num, name) — %d for int, %s for String, %f for double/float.

3. Input (Scanner)

```
import java.util.Scanner;  
Scanner sc = new Scanner(System.in);  
int a = sc.nextInt();  
double d = sc.nextDouble();  
String s = sc.nextLine();  
char c = sc.next().charAt(0);  
boolean b = sc.nextBoolean();
```

Common mistake: nextInt()/nextDouble() leave a newline in the buffer, so a following nextLine() reads empty. Add an extra sc.nextLine() to consume it before reading a full line.

4. Variables & Data Types

Type	Size	Range / Use
int	4 bytes	Whole numbers, most common integer type
long	8 bytes	Very large whole numbers (suffix with L)
double	8 bytes	Decimal numbers, default for floating point

float	4 bytes	Decimal numbers, less precision (suffix with f)
char	2 bytes	Single character, stored as Unicode
boolean	1 bit	true / false only
byte	1 byte	Small whole numbers, -128 to 127
short	2 bytes	Whole numbers, -32768 to 32767
String	—	Not primitive — it's a class/object, immutable

5. Type Casting

Implicit (widening) — automatic, no data loss: byte → short → int → long → float → double

```
int a = 10; double b = a; // b = 10.0, automatic
```

Explicit (narrowing) — manual, can lose data: double → float → long → int → short → byte

```
double d = 9.7; int x = (int) d; // x = 9 — decimal truncated, NOT rounded
int big = 300; byte small = (byte) big; // overflow — gives unexpected value (44)
```

Common mistake: assuming (int) rounds off. It doesn't — it always truncates toward zero. Use Math.round() if you actually want rounding.

6. Operators

Arithmetic: + - * / % (modulo gives remainder)

Relational: == != > < >= <= (all return boolean)

Logical: && (AND) || (OR) ! (NOT)

Assignment: = += -= *= /= %=

Unary: ++ (increment) -- (decrement) - (negation) — pre vs post matters:

```
int a = 5;
int b = a++; // b = 5, then a becomes 6 (post: use old value first)
int c = ++a; // a becomes 7 first, then c = 7 (pre: increment first)
```

Ternary: condition ? valueIfTrue : valueIfFalse — a compact if-else

```
int max = (a > b) ? a : b;
```

Common mistake: using = instead of == in a comparison. Java blocks this for boolean contexts, but the habit causes bugs in other languages — still worth being careful.

Integer division trap: 5 / 2 gives 2, not 2.5, because both operands are int. Use 5.0 / 2 or (double) 5 / 2 to force a decimal result.

7. Operator Precedence (high to low, common ones)

1. () parentheses → 2. ++ -- (unary) → 3. * / % → 4. + - → 5. < <= > >= → 6. == != → 7. && → 8. || → 9. ?: (ternary) → 10. = += -= etc.

Example trap: $10 + 5 > 12 \ \&\& \ 3 < 4$ — arithmetic and relational happen before $\&\&$, so this evaluates as $(15 > 12) \ \&\& \ (3 < 4) = \text{true}$.

8. Math Class Functions

Method	Use
<code>Math.sqrt(x)</code>	Square root of x
<code>Math.pow(a, b)</code>	a raised to the power b
<code>Math.abs(x)</code>	Absolute value (removes negative sign)
<code>Math.max(a, b)</code>	Larger of two values
<code>Math.min(a, b)</code>	Smaller of two values
<code>Math.round(x)</code>	Rounds to nearest whole number
<code>Math.ceil(x)</code>	Rounds up to nearest whole number (returns double)
<code>Math.floor(x)</code>	Rounds down to nearest whole number (returns double)
<code>Math.PI</code>	Constant 3.14159... (not a method, a field)

Common mistake: forgetting `Math.pow()` always returns a double — even `Math.pow(2,3)` returns 8.0, not 8. Cast to (int) if you need a whole number.

9. Escape Sequences

`\n` = new line | `\t` = tab | `\"` = double quote | `\\` = backslash | `'` = single quote

10. Comments & Constants

`//` single-line comment | `/*` multi-line comment `*/`

`final int MAX = 100;` `// 'final'` makes it a constant — cannot be reassigned after this

Quick Revision Checklist

- Can I write the `main()` method structure from memory?
- Do I know which Scanner method to use for int / double / String / char / boolean?
- Can I explain why $5/2 = 2$ but $5.0/2 = 2.5$?
- Do I know implicit vs explicit casting, and that (int) truncates, not rounds?
- Can I explain pre-increment vs post-increment with an example?
- Do I remember that `Math.pow()` returns a double, always?
- Can I recall operator precedence for at least arithmetic vs relational vs logical?

Part 2 — Practice Questions

Covers every subtopic above: I/O, variables, casting, all operator types, Math functions, and precedence — so no variety is missed.

Easy (5)

- 1 Print your name, age, and college name using `System.out.println()`.
- 2 Take two integers as input and print their sum, difference, product, quotient, and remainder.
- 3 Take the radius of a circle as input and print its area (use `Math.PI`) and circumference.
- 4 Take a double value and cast it to int; print both values to show the truncation (not rounding).
- 5 Take two numbers as input and print the larger one using the ternary operator.

Medium (5)

- 1 Take a number as input, use `Math.pow()` to print its square and cube, and cast the result to int.
- 2 Take the length and breadth of a rectangle and print area, perimeter, and diagonal using `Math.sqrt()`.
- 3 Take an integer as input, demonstrate pre-increment and post-increment on it, and print both results with explanation comments.
- 4 Take a decimal number as input and print its `Math.ceil()`, `Math.floor()`, and `Math.round()` values, explaining the difference.
- 5 Take principal, rate, and time as input and calculate simple interest and compound interest (use `Math.pow()` for compound interest).

Hard (5)

- 1 Take two int variables and swap their values without a third variable, using arithmetic operators only (no casting bugs allowed).
- 2 Take a large int value (e.g., 300) and narrow-cast it to byte; print the unexpected overflowed result and explain why it happened.
- 3 Take a weight (kg) and height (m), calculate BMI using `Math.pow()` for height squared, and print the result formatted to 2 decimal places using `printf`.
- 4 Given three numbers, find the maximum using nested ternary operators only (no if-else).
- 5 Write an expression using at least 4 different operator types (arithmetic, relational, logical, ternary) in one line, evaluate it by hand first, then verify with code — to test operator precedence understanding.

How to use this: Read Part 1 fully first — don't skip Math functions or casting, they show up in the questions. Then solve Part 2 in order. If any question feels unfamiliar, it means a notes section above needs a re-read before moving to If-Else.